Java One-Stop Regular Problems and Solutions

Collections, Streams, Optional, Cache, Threads, Virtual Threads, Runnable, and core DSA/Algo in one guide.

This is a practical companion to tricky output questions. These are regular coding problems with solution patterns and Java implementations.

How to Use This Guide

1. Solve by section first.
2. Time-box each problem (20-35 min).
3. Re-implement without looking.
4. Revisit with complexity constraints.

Section A: Collections Problems (P1-P12)

P1. Two Sum using HashMap

Problem: Return indices of two numbers that add to target.

```
int[] twoSum(int[] nums, int target) {
  Map<Integer, Integer> seen = new HashMap<>();
  for (int i = 0; i < nums.length; i++) {
    int need = target - nums[i];
    if (seen.containsKey(need)) return new int[]{seen.get(need), i};
    seen.put(nums[i], i);
  }
  return new int[]{-1, -1};
}
```

Time: O(n), Space: O(n)

P2. Group Anagrams

Problem: Group strings that are anagrams.

```
List<List<String>> groupAnagrams(String[] strs) {
  Map<String, List<String>> map = new HashMap<>();
  for (String s : strs) {
    char[] c = s.toCharArray();
    Arrays.sort(c);
    String key = new String(c);
    map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
  }
  return new ArrayList<>(map.values());
}
```

Time: O(n * k log k)

P3. Top K Frequent Elements

```java
int[] topKFrequent(int[] nums, int k) {
  Map<Integer, Integer> freq = new HashMap<>();
  for (int n : nums) freq.merge(n, 1, Integer::sum);

  PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[1]));
  for (var e : freq.entrySet()) {
    pq.offer(new int[]{e.getKey(), e.getValue()});
    if (pq.size() > k) pq.poll();
  }

  int[] ans = new int[k];
  for (int i = k - 1; i >= 0; i--) ans[i] = pq.poll()[0];
  return ans;
}
```

Time: O(n log k)

P4. Merge Intervals

```java
int[][] merge(int[][] intervals) {
  Arrays.sort(intervals, Comparator.comparingInt(a -> a[0]));
  List<int[]> out = new ArrayList<>();
  for (int[] cur : intervals) {
    if (out.isEmpty() || out.get(out.size() - 1)[1] < cur[0]) out.add(cur);
    else out.get(out.size() - 1)[1] = Math.max(out.get(out.size() - 1)[1], cur[1]);
  }
  return out.toArray(new int[0][]);
}
```

P5. Longest Consecutive Sequence

```java
int longestConsecutive(int[] nums) {
  Set<Integer> set = new HashSet<>();
  for (int n : nums) set.add(n);
  int best = 0;
  for (int n : set) {
    if (!set.contains(n - 1)) {
      int cur = n, len = 1;
      while (set.contains(cur + 1)) { cur++; len++; }
      best = Math.max(best, len);
    }
  }
  return best;
}
```

P6. LRU Cache (LinkedHashMap)

```
class LRUCache<K, V> extends LinkedHashMap<K, V> {
  private final int capacity;

  LRUCache(int capacity) {
    super(16, 0.75f, true);
    this.capacity = capacity;
  }

  @Override
  protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
    return size() > capacity;
  }
}
```

P7. Design Min Stack

```
class MinStack {
  Deque<Integer> stack = new ArrayDeque<>();
  Deque<Integer> mins = new ArrayDeque<>();

  void push(int x) {
    stack.push(x);
    if (mins.isEmpty() || x <= mins.peek()) mins.push(x);
  }

  void pop() {
    int x = stack.pop();
    if (x == mins.peek()) mins.pop();
  }

  int top() { return stack.peek(); }
  int getMin() { return mins.peek(); }
}
```

P8. Sliding Window Maximum (Deque)

```
int[] maxSlidingWindow(int[] nums, int k) {
  Deque<Integer> dq = new ArrayDeque<>();
  int[] ans = new int[nums.length - k + 1];
  int idx = 0;

  for (int i = 0; i < nums.length; i++) {
    while (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst();
    while (!dq.isEmpty() && nums[dq.peekLast()] <= nums[i]) dq.pollLast();
    dq.offerLast(i);
    if (i >= k - 1) ans[idx++] = nums[dq.peekFirst()];
  }
```

```
  return ans;
}
```

P9. K Closest Points to Origin

```
int[][] kClosest(int[][] points, int k) {
  PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) ->
      Integer.compare(dist(b), dist(a)));

  for (int[] p : points) {
    pq.offer(p);
    if (pq.size() > k) pq.poll();
  }
  return pq.toArray(new int[0][]);
}

int dist(int[] p) { return p[0] * p[0] + p[1] * p[1]; }
```

P10. Find Duplicate Files by Content (Map of List)

```
Map<String, List<String>> groupByContent(List<String[]> files) {
  Map<String, List<String>> byContent = new HashMap<>();
  for (String[] file : files) {
    String path = file[0], content = file[1];
    byContent.computeIfAbsent(content, c -> new ArrayList<>()).add(path);
  }
  return byContent;
}
```

P11. Sort Map by Value Desc

```
LinkedHashMap<String, Integer> sortByValueDesc(Map<String, Integer> input) {
  return input.entrySet().stream()
      .sorted((a, b) -> Integer.compare(b.getValue(), a.getValue()))
      .collect(LinkedHashMap::new,
          (m, e) -> m.put(e.getKey(), e.getValue()),
          Map::putAll);
}
```

P12. Frequency Sort Characters

```
String frequencySort(String s) {
  Map<Character, Integer> freq = new HashMap<>();
  for (char c : s.toCharArray()) freq.merge(c, 1, Integer::sum);

  List<Character> chars = new ArrayList<>(freq.keySet());
  chars.sort((a, b) -> Integer.compare(freq.get(b), freq.get(a)));
```

```
  StringBuilder sb = new StringBuilder();
  for (char c : chars) {
    sb.append(String.valueOf(c).repeat(freq.get(c)));
  }
  return sb.toString();
}
```

Section B: Streams Problems (P13-P22)

P13. Sum Even Numbers

```
int sumEven(List<Integer> nums) {
  return nums.stream().filter(n -> n % 2 == 0).mapToInt(Integer::intValue).sum();
}
```

P14. Distinct Sorted Strings

```
List<String> distinctSorted(List<String> in) {
  return in.stream().distinct().sorted().toList();
}
```

P15. Partition by Predicate

```
Map<Boolean, List<Integer>> partitionEvenOdd(List<Integer> nums) {
  return nums.stream().collect(Collectors.partitioningBy(n -> n % 2 == 0));
}
```

P16. Group Employees by Department

```
record Employee(String name, String dept, int salary) {}

Map<String, List<Employee>> byDept(List<Employee> emps) {
  return emps.stream().collect(Collectors.groupingBy(Employee::dept));
}
```

P17. Max Salary per Department

```
Map<String, Optional<Employee>> maxSalaryByDept(List<Employee> emps) {
  return emps.stream().collect(Collectors.groupingBy(
      Employee::dept,
      Collectors.maxBy(Comparator.comparingInt(Employee::salary))
  ));
}
```

P18. Convert List to Map with Merge Function

```
Map<String, Integer> toMapWithMerge(List<Employee> emps) {
  return emps.stream().collect(Collectors.toMap(
      Employee::dept,
      Employee::salary,
      Integer::max
  ));
}
```

P19. Flatten Nested Lists

```
List<Integer> flatten(List<List<Integer>> nested) {
  return nested.stream().flatMap(List::stream).toList();
}
```

P20. Windowed Pair Sum (imperative when streams hurt readability)

```
List<Integer> pairSums(int[] a) {
  List<Integer> out = new ArrayList<>();
  for (int i = 0; i + 1 < a.length; i++) out.add(a[i] + a[i + 1]);
  return out;
}
```

P21. Safe Parallel Sum

```
long parallelSum(List<Integer> nums) {
  return nums.parallelStream().mapToLong(Integer::longValue).sum();
}
```

P22. Top N by Stream + Heap

```
List<Integer> topN(List<Integer> nums, int n) {
  PriorityQueue<Integer> pq = new PriorityQueue<>();
  nums.forEach(x -> {
    pq.offer(x);
    if (pq.size() > n) pq.poll();
  });
  List<Integer> out = new ArrayList<>(pq);
  out.sort(Comparator.reverseOrder());
  return out;
}
```

Section C: Optional Problems (P23-P28)

P23. Parse Integer Safely

```
Optional<Integer> safeParseInt(String s) {
  try {
    return Optional.of(Integer.parseInt(s));
  } catch (NumberFormatException e) {
    return Optional.empty();
  }
}
```

P24. Optional Chaining for Nested Object

```
record Address(String city) {}
record User(Address address) {}

String cityOrUnknown(User u) {
  return Optional.ofNullable(u)
      .map(User::address)
      .map(Address::city)
      .orElse("UNKNOWN");
}
```

P25. Throw Domain Exception if Missing

```
String requireToken(Optional<String> token) {
  return token.filter(t -> !t.isBlank())
      .orElseThrow(() -> new IllegalArgumentException("Missing token"));
}
```

P26. Avoid Optional in Fields (use nullable field + Optional getter)

```
class Config {
  private String region;
  Optional<String> region() { return Optional.ofNullable(region); }
}
```

P27. Optional + Stream

```
List<String> normalize(Optional<String> a, Optional<String> b) {
  return Stream.of(a, b)
      .flatMap(Optional::stream)
      .map(String::trim)
      .filter(s -> !s.isEmpty())
      .toList();
}
```

P28. Optional Fallback with Supplier

```
String loadConfig(Optional<String> env, Supplier<String> fileLoader) {
  return env.orElseGet(fileLoader);
}
```

Section D: Cache Problems (P29-P36)

P29. Simple Read-Through Cache

```
class ReadThroughCache<K, V> {
  private final Map<K, V> cache = new ConcurrentHashMap<>();

  V get(K key, Function<K, V> loader) {
    return cache.computeIfAbsent(key, loader);
  }
}
```

P30. Cache with TTL

```
class TtlCache<K, V> {
  static class Entry<V> {
    final V value;
    final long expiresAt;
    Entry(V value, long expiresAt) { this.value = value; this.expiresAt = expiresAt; }
  }

  private final Map<K, Entry<V>> map = new ConcurrentHashMap<>();

  V get(K key, Supplier<V> loader, long ttlMs) {
    Entry<V> e = map.get(key);
    long now = System.currentTimeMillis();
    if (e != null && e.expiresAt > now) return e.value;
    V v = loader.get();
    map.put(key, new Entry<>(v, now + ttlMs));
    return v;
  }
}
```

P31. Prevent Cache Stampede with CompletableFuture

```
class AsyncCache<K, V> {
  private final ConcurrentHashMap<K, CompletableFuture<V>> inFlight = new
ConcurrentHashMap<>();

  CompletableFuture<V> get(K key, Supplier<V> loader) {
    return inFlight.computeIfAbsent(key, k -> CompletableFuture.supplyAsync(() -> {
```

```
      try { return loader.get(); }
      finally { inFlight.remove(k); }
    }));
  }
}
```

P32. Negative Caching

```
record Result(Optional<String> value) {}

Result findOrNegativeCache(String key, Map<String, Result> cache, Function<String,
Optional<String>> source) {
  return cache.computeIfAbsent(key, k -> new Result(source.apply(k)));
}
```

P33. Size-based LRU with LinkedHashMap

```
class LruMap<K, V> extends LinkedHashMap<K, V> {
  private final int max;
  LruMap(int max) { super(16, 0.75f, true); this.max = max; }
  @Override protected boolean removeEldestEntry(Map.Entry<K, V> e) { return size() > max; }
}
```

P34. Tiny LFU-Style Frequency Counter

```
class FrequencyCounter<K> {
  private final Map<K, LongAdder> hits = new ConcurrentHashMap<>();
  void hit(K key) { hits.computeIfAbsent(key, k -> new LongAdder()).increment(); }
  long count(K key) { return hits.getOrDefault(key, new LongAdder()).sum(); }
}
```

P35. Semaphore-protected Downstream Calls

```
class LimitedClient {
  private final Semaphore sem = new Semaphore(20);

  <T> T call(Callable<T> c) throws Exception {
    sem.acquire();
    try { return c.call(); }
    finally { sem.release(); }
  }
}
```

P36. Cache Invalidation by Version Token

```
class VersionedCache<K, V> {
  record KeyWithVersion<K>(K key, long version) {}
  private final Map<KeyWithVersion<K>, V> map = new ConcurrentHashMap<>();

  V put(K key, long version, V value) {
    return map.put(new KeyWithVersion<>(key, version), value);
  }
}
```

Section E: Threads, Runnable, Virtual Threads, and Concurrency (P37-P48)

P37. Runnable vs Thread

```
Runnable task = () -> System.out.println("work");
new Thread(task).start();
```

P38. Correct Shared Counter with AtomicInteger

```
AtomicInteger counter = new AtomicInteger();
counter.incrementAndGet();
```

P39. Producer Consumer with BlockingQueue

```
BlockingQueue<Integer> q = new ArrayBlockingQueue<>(100);

Runnable producer = () -> {
  for (int i = 0; i < 1000; i++) {
    try { q.put(i); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
  }
};

Runnable consumer = () -> {
  while (!Thread.currentThread().isInterrupted()) {
    try { q.take(); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
  }
};
```

P40. Parallel API Calls with CompletableFuture

```
CompletableFuture<String> f1 = CompletableFuture.supplyAsync(() -> "A");
CompletableFuture<String> f2 = CompletableFuture.supplyAsync(() -> "B");
String out = f1.thenCombine(f2, (a, b) -> a + b).join();
```

P41. Timeout on Async Task

```
String result = CompletableFuture.supplyAsync(() -> slowCall())
    .orTimeout(500, TimeUnit.MILLISECONDS)
    .exceptionally(ex -> "fallback")
    .join();
```

P42. Virtual Thread per Task (Java 21+)

```
try (var ex = Executors.newVirtualThreadPerTaskExecutor()) {
  Future<Integer> f = ex.submit(() -> 42);
  System.out.println(f.get());
}
```

P43. Fan-out/Fan-in with Virtual Threads

```
String fetchBoth() throws Exception {
  try (var ex = Executors.newVirtualThreadPerTaskExecutor()) {
    Future<String> a = ex.submit(() -> serviceA());
    Future<String> b = ex.submit(() -> serviceB());
    return a.get() + ":" + b.get();
  }
}
```

P44. Limit Virtual Concurrency with Semaphore (not thread pool)

```
Semaphore sem = new Semaphore(50);

String guardedCall() throws Exception {
  sem.acquire();
  try { return downstream(); }
  finally { sem.release(); }
}
```

P45. Structured Task Scope (Java 21 preview APIs may vary)

```
// Pseudocode-style interview answer:
// fork(taskA), fork(taskB), join(), throwIfFailed(), combine results.
// Mention deterministic cancellation and better observability.
```

P46. Avoid Pinning Hot Paths

```
ReentrantLock lock = new ReentrantLock();

void criticalIo() {
```

```
  lock.lock();
  try {
    // do short critical update, avoid long blocking I/O while locked
  } finally {
    lock.unlock();
  }
}
```

P47. Graceful Shutdown of Executor

```
void stop(ExecutorService ex) {
  ex.shutdown();
  try {
    if (!ex.awaitTermination(5, TimeUnit.SECONDS)) ex.shutdownNow();
  } catch (InterruptedException e) {
    ex.shutdownNow();
    Thread.currentThread().interrupt();
  }
}
```

P48. Deadlock Detection Thinking

```
// Interview answer:
// 1) fixed lock order
// 2) tryLock with timeout
// 3) small critical sections
// 4) jstack/jcmd/JFR for diagnosis
```

Section F: Core DSA/Algo Problems in Java (P49-P75)

P49. Binary Search

```
int bs(int[] a, int t) {
  int l = 0, r = a.length - 1;
  while (l <= r) {
    int m = l + (r - l) / 2;
    if (a[m] == t) return m;
    if (a[m] < t) l = m + 1;
    else r = m - 1;
  }
  return -1;
}
```

P50. First Bad Version Pattern

```
int firstTrue(IntPredicate pred, int n) {
  int l = 1, r = n;
  while (l < r) {
    int m = l + (r - l) / 2;
    if (pred.test(m)) r = m;
    else l = m + 1;
  }
  return l;
}
```

P51. Two Pointers: Valid Palindrome

```
boolean isPalindrome(String s) {
  int l = 0, r = s.length() - 1;
  while (l < r) {
    while (l < r && !Character.isLetterOrDigit(s.charAt(l))) l++;
    while (l < r && !Character.isLetterOrDigit(s.charAt(r))) r--;
    if (Character.toLowerCase(s.charAt(l++)) != Character.toLowerCase(s.charAt(r--))) return
false;
  }
  return true;
}
```

P52. Sliding Window: Longest Substring Without Repeating

```
int lengthOfLongestSubstring(String s) {
  Map<Character, Integer> last = new HashMap<>();
  int l = 0, best = 0;
  for (int r = 0; r < s.length(); r++) {
    char c = s.charAt(r);
    if (last.containsKey(c)) l = Math.max(l, last.get(c) + 1);
    last.put(c, r);
    best = Math.max(best, r - l + 1);
  }
  return best;
}
```

P53. Prefix Sum: Subarray Sum Equals K

```
int subarraySum(int[] nums, int k) {
  Map<Integer, Integer> cnt = new HashMap<>();
  cnt.put(0, 1);
  int sum = 0, ans = 0;
  for (int x : nums) {
    sum += x;
    ans += cnt.getOrDefault(sum - k, 0);
    cnt.merge(sum, 1, Integer::sum);
  }
```

```
  return ans;
}
```

P54. Monotonic Stack: Next Greater Element

```
int[] nextGreater(int[] nums) {
  int n = nums.length;
  int[] ans = new int[n];
  Arrays.fill(ans, -1);
  Deque<Integer> st = new ArrayDeque<>();
  for (int i = 0; i < n; i++) {
    while (!st.isEmpty() && nums[st.peek()] < nums[i]) ans[st.pop()] = nums[i];
    st.push(i);
  }
  return ans;
}
```

P55. Kadane Maximum Subarray

```
int maxSubArray(int[] nums) {
  int best = nums[0], cur = nums[0];
  for (int i = 1; i < nums.length; i++) {
    cur = Math.max(nums[i], cur + nums[i]);
    best = Math.max(best, cur);
  }
  return best;
}
```

P56. Merge K Sorted Lists (Min Heap)

```
class ListNode { int val; ListNode next; ListNode(int v) { val = v; } }

ListNode mergeKLists(ListNode[] lists) {
  PriorityQueue<ListNode> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a.val));
  for (ListNode n : lists) if (n != null) pq.offer(n);
  ListNode dummy = new ListNode(0), tail = dummy;
  while (!pq.isEmpty()) {
    ListNode cur = pq.poll();
    tail.next = cur;
    tail = tail.next;
    if (cur.next != null) pq.offer(cur.next);
  }
  return dummy.next;
}
```

P57. BFS Shortest Path in Unweighted Graph

```
int shortestPath(List<List<Integer>> g, int src, int dst) {
  int n = g.size();
  int[] dist = new int[n];
  Arrays.fill(dist, -1);
  Queue<Integer> q = new ArrayDeque<>();
  q.offer(src);
  dist[src] = 0;
  while (!q.isEmpty()) {
    int u = q.poll();
    if (u == dst) return dist[u];
    for (int v : g.get(u)) if (dist[v] == -1) {
      dist[v] = dist[u] + 1;
      q.offer(v);
    }
  }
  return -1;
}
```

P58. DFS Cycle Detection in Directed Graph

```
boolean hasCycle(List<List<Integer>> g) {
  int n = g.size();
  int[] state = new int[n];
  for (int i = 0; i < n; i++) if (dfs(i, g, state)) return true;
  return false;
}

boolean dfs(int u, List<List<Integer>> g, int[] st) {
  if (st[u] == 1) return true;
  if (st[u] == 2) return false;
  st[u] = 1;
  for (int v : g.get(u)) if (dfs(v, g, st)) return true;
  st[u] = 2;
  return false;
}
```

P59. Topological Sort (Kahn)

```
List<Integer> topo(int n, int[][] edges) {
  List<List<Integer>> g = new ArrayList<>();
  for (int i = 0; i < n; i++) g.add(new ArrayList<>());
  int[] indeg = new int[n];
  for (int[] e : edges) { g.get(e[0]).add(e[1]); indeg[e[1]]++; }

  Queue<Integer> q = new ArrayDeque<>();
  for (int i = 0; i < n; i++) if (indeg[i] == 0) q.offer(i);

  List<Integer> out = new ArrayList<>();
  while (!q.isEmpty()) {
```

```
    int u = q.poll();
    out.add(u);
    for (int v : g.get(u)) if (--indeg[v] == 0) q.offer(v);
  }
  return out.size() == n ? out : List.of();
}
```

P60. Dijkstra

```
int[] dijkstra(List<List<int[]>> g, int src) {
  int n = g.size();
  int[] dist = new int[n];
  Arrays.fill(dist, Integer.MAX_VALUE);
  dist[src] = 0;
  PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[1]));
  pq.offer(new int[]{src, 0});

  while (!pq.isEmpty()) {
    int[] cur = pq.poll();
    int u = cur[0], d = cur[1];
    if (d != dist[u]) continue;
    for (int[] e : g.get(u)) {
      int v = e[0], w = e[1];
      if (dist[u] + w < dist[v]) {
        dist[v] = dist[u] + w;
        pq.offer(new int[]{v, dist[v]});
      }
    }
  }
  return dist;
}
```

P61. Union Find

```
class DSU {
  int[] p, r;
  DSU(int n) {
    p = new int[n]; r = new int[n];
    for (int i = 0; i < n; i++) p[i] = i;
  }
  int find(int x) { return p[x] == x ? x : (p[x] = find(p[x])); }
  boolean union(int a, int b) {
    int ra = find(a), rb = find(b);
    if (ra == rb) return false;
    if (r[ra] < r[rb]) p[ra] = rb;
    else if (r[ra] > r[rb]) p[rb] = ra;
    else { p[rb] = ra; r[ra]++; }
    return true;
```

```
  }
}
```

P62. 0/1 Knapsack

```
int knapsack(int[] w, int[] v, int cap) {
  int[] dp = new int[cap + 1];
  for (int i = 0; i < w.length; i++) {
    for (int c = cap; c >= w[i]; c--) {
      dp[c] = Math.max(dp[c], dp[c - w[i]] + v[i]);
    }
  }
  return dp[cap];
}
```

P63. Coin Change (min coins)

```
int coinChange(int[] coins, int amount) {
  int INF = amount + 1;
  int[] dp = new int[amount + 1];
  Arrays.fill(dp, INF);
  dp[0] = 0;
  for (int a = 1; a <= amount; a++) {
    for (int c : coins) if (a - c >= 0) dp[a] = Math.min(dp[a], dp[a - c] + 1);
  }
  return dp[amount] == INF ? -1 : dp[amount];
}
```

P64. LIS O(n log n)

```
int lengthOfLIS(int[] nums) {
  int[] tails = new int[nums.length];
  int size = 0;
  for (int x : nums) {
    int i = Arrays.binarySearch(tails, 0, size, x);
    if (i < 0) i = -(i + 1);
    tails[i] = x;
    if (i == size) size++;
  }
  return size;
}
```

P65. Trie Insert/Search

```
class Trie {
  static class Node { Node[] next = new Node[26]; boolean end; }
  Node root = new Node();
```

```
  void insert(String s) {
    Node cur = root;
    for (char c : s.toCharArray()) {
      int i = c - 'a';
      if (cur.next[i] == null) cur.next[i] = new Node();
      cur = cur.next[i];
    }
    cur.end = true;
  }

  boolean search(String s) {
    Node cur = root;
    for (char c : s.toCharArray()) {
      int i = c - 'a';
      if (cur.next[i] == null) return false;
      cur = cur.next[i];
    }
    return cur.end;
  }
}
```

P66. Backtracking: Subsets

```
List<List<Integer>> subsets(int[] nums) {
  List<List<Integer>> ans = new ArrayList<>();
  backtrack(0, nums, new ArrayList<>(), ans);
  return ans;
}

void backtrack(int i, int[] nums, List<Integer> cur, List<List<Integer>> out) {
  if (i == nums.length) {
    out.add(new ArrayList<>(cur));
    return;
  }
  backtrack(i + 1, nums, cur, out);
  cur.add(nums[i]);
  backtrack(i + 1, nums, cur, out);
  cur.remove(cur.size() - 1);
}
```

P67. Backtracking: Permutations

```
List<List<Integer>> permute(int[] nums) {
  List<List<Integer>> ans = new ArrayList<>();
  boolean[] used = new boolean[nums.length];
  dfs(nums, used, new ArrayList<>(), ans);
  return ans;
}
```

```
void dfs(int[] nums, boolean[] used, List<Integer> cur, List<List<Integer>> out) {
  if (cur.size() == nums.length) {
    out.add(new ArrayList<>(cur));
    return;
  }
  for (int i = 0; i < nums.length; i++) {
    if (used[i]) continue;
    used[i] = true;
    cur.add(nums[i]);
    dfs(nums, used, cur, out);
    cur.remove(cur.size() - 1);
    used[i] = false;
  }
}
```

P68. Binary Tree Level Order

```
class TreeNode { int val; TreeNode left, right; TreeNode(int v) { val = v; } }

List<List<Integer>> levelOrder(TreeNode root) {
  List<List<Integer>> ans = new ArrayList<>();
  if (root == null) return ans;
  Queue<TreeNode> q = new ArrayDeque<>();
  q.offer(root);
  while (!q.isEmpty()) {
    int sz = q.size();
    List<Integer> level = new ArrayList<>();
    for (int i = 0; i < sz; i++) {
      TreeNode n = q.poll();
      level.add(n.val);
      if (n.left != null) q.offer(n.left);
      if (n.right != null) q.offer(n.right);
    }
    ans.add(level);
  }
  return ans;
}
```

P69. Lowest Common Ancestor in BST

```
TreeNode lca(TreeNode root, TreeNode p, TreeNode q) {
  while (root != null) {
    if (p.val < root.val && q.val < root.val) root = root.left;
    else if (p.val > root.val && q.val > root.val) root = root.right;
    else return root;
  }
  return null;
}
```

P70. Reverse Linked List

```
ListNode reverse(ListNode head) {
  ListNode prev = null, cur = head;
  while (cur != null) {
    ListNode next = cur.next;
    cur.next = prev;
    prev = cur;
    cur = next;
  }
  return prev;
}
```

P71. Detect Cycle in Linked List

```
boolean hasCycle(ListNode head) {
  ListNode slow = head, fast = head;
  while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow == fast) return true;
  }
  return false;
}
```

P72. Valid Parentheses

```
boolean isValid(String s) {
  Map<Character, Character> map = Map.of(')', '(', ']', '[', '}', '{');
  Deque<Character> st = new ArrayDeque<>();
  for (char c : s.toCharArray()) {
    if (map.containsValue(c)) st.push(c);
    else if (map.containsKey(c)) {
      if (st.isEmpty() || st.pop() != map.get(c)) return false;
    }
  }
  return st.isEmpty();
}
```

P73. Search in Rotated Sorted Array

```
int searchRotated(int[] nums, int target) {
  int l = 0, r = nums.length - 1;
  while (l <= r) {
    int m = l + (r - l) / 2;
    if (nums[m] == target) return m;
    if (nums[l] <= nums[m]) {
```

```
      if (nums[l] <= target && target < nums[m]) r = m - 1;
      else l = m + 1;
    } else {
      if (nums[m] < target && target <= nums[r]) l = m + 1;
      else r = m - 1;
    }
  }
  return -1;
}
```

P74. Median of Data Stream (Two Heaps)

```
class MedianFinder {
  PriorityQueue<Integer> lo = new PriorityQueue<>(Comparator.reverseOrder());
  PriorityQueue<Integer> hi = new PriorityQueue<>();

  void addNum(int x) {
    if (lo.isEmpty() || x <= lo.peek()) lo.offer(x);
    else hi.offer(x);

    if (lo.size() > hi.size() + 1) hi.offer(lo.poll());
    if (hi.size() > lo.size()) lo.offer(hi.poll());
  }

  double findMedian() {
    if (lo.size() == hi.size()) return (lo.peek() + hi.peek()) / 2.0;
    return lo.peek();
  }
}
```

P75. Word Break

```
boolean wordBreak(String s, List<String> wordDict) {
  Set<String> set = new HashSet<>(wordDict);
  boolean[] dp = new boolean[s.length() + 1];
  dp[0] = true;
  for (int i = 1; i <= s.length(); i++) {
    for (int j = 0; j < i; j++) {
      if (dp[j] && set.contains(s.substring(j, i))) {
        dp[i] = true;
        break;
      }
    }
  }
  return dp[s.length()];
}
```

Interview-Grade Discussion Topics (What to Say)

- Collections:
  - HashMap vs ConcurrentHashMap tradeoffs.
  - ArrayList vs LinkedList in real workloads.
  - Why TreeMap gives ordered keys.
- Streams:
  - Lazy pipeline, terminal ops, and side-effect cautions.
  - Why parallel streams are workload-sensitive.
- Optional:
  - Great for return types, avoid as fields/params in most codebases.
- Cache:
  - TTL, invalidation, stampede control, and memory limits.
- Threads and Virtual Threads:
  - Virtual threads improve throughput for blocking I/O workloads.
  - Do not pool virtual threads; use semaphore for limits.
  - Avoid long blocking I/O while pinned in synchronized sections.
- DSA/Algo:
  - Pattern recognition first: sliding window, BFS/DFS, heap, DP, union-find.

Internet-Backed Reference Pointers

These topics were expanded using official Java platform guidance and commonly accepted interview patterns:

1. Virtual Threads (Oracle Java 21 docs): https://docs.oracle.com/en/java/javase/21/core/virtual-threads.html
2. JEP 444 Virtual Threads (OpenJDK): https://openjdk.org/jeps/444
3. Stream API package summary: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/stream/package-summary.html
4. Optional API docs: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Optional.html
5. Caffeine design notes (popular Java caching reference): https://github.com/ben-manes/caffeine/wiki

Use these references for deep API semantics; this guide stays interview-practical and solution-oriented.